

Document 01 — Python & Engineering Foundations

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

None. Start here.

How to Read & Practice This Document

- **What:** solid Python basics — typed functions, config loading, logging, handling errors properly.
- **Why:** every later document assumes you know this. Skip it, and future bugs will feel confusing when they're really just a missing type hint or a swallowed error.
- **When:** you'll use this in every script from here on.
- **How to practice:** 1. Read the material once, just to get the shape of it. Don't try to memorize it. 2. Do the **Basic** exercises with no notes open. This shows you what you actually remember. 3. Do the **Intermediate/Real-world** exercises with notes open — that's normal, not cheating. 4. Try the **Build Task** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use the hint system (**Hint 1** → **Hint 4**) only after you've really tried. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, explain out loud (or write a paragraph) what you built and why. If you can't, you're not done yet.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Picture this: you're about to build your first AI agent. Before it can do anything smart, it needs three boring but critical things to be right — or it will fail in confusing ways later.

First, it needs a secret key (like `OPENAI_API_KEY`) to talk to an AI service. That key can never be typed directly into your code, because code gets shared, backed up, and pushed to git — and a leaked key is a real security incident. So the key lives in a separate `.env` file on your own computer, and your program reads it at startup.

Second, your program needs to check that the key (and any other required setting) actually showed up. If it's missing, you want your program to stop immediately with a clear message — "OPENAI_API_KEY is missing" — not five minutes later with a confusing error from OpenAI's servers. This is what a **custom error** and a **config loader** are for: they turn "something vague went wrong somewhere" into "this exact thing is missing, right here, right now."

Third, once your program is running, you need to know what it's doing — especially later, when it's running unattended (a server, a scheduled job) and something breaks while you're not watching.

`print()` isn't enough for that. A **logger** keeps a running record you can check afterward, and can be as quiet or as detailed as you want, without touching the code.

That's the whole story of this document: **type hints and custom errors** teach you to build a function that fails loudly and clearly instead of silently and confusingly. **Virtual environments** keep this project's tools separate from every other project on your computer. **.env files** keep your secrets out of your code. **Logging** gives you a record of what happened. And the **Build Task** at the end asks you to put all four of these together into one small, real thing: a config loader and a logger that any later project in this whole curriculum can reuse as its starting point. Every project you build from here on — a chatbot, a multi-agent system, anything — starts by copying this exact Build Task's two files.

Core Concepts (read this first — everything you need is here)

Functions, types, and why they matter

A function is a named, reusable piece of code — but the part that matters most for real projects (not just quick scripts) is the **signature**: what goes in, what comes out, and what it promises never to do. `def load_config() -> Config:` tells any reader — including you, months later — exactly what to expect, without reading the whole function. Without types (`def load_config():`), the code still runs the same, but now every reader has to guess or dig through the code to understand it. **Why this matters**: once more than one file uses another file's code, mismatched guesses about what a function does become the most common source of bugs — not bad logic, but a misunderstanding about the *interface*. **When to bother with types**: always, in any code meant to last longer than a script you'll delete tomorrow. **How it works**: Python doesn't actually check type hints while your program runs — tools like `mypy` or your editor read them and warn you *before* you run the code. A wrong type hint won't crash your program; it just stops protecting you, so keep your hints accurate.

Errors: a clean way to say "something specific went wrong"

An error (called an "exception" in Python) is how your code stops normal execution when something breaks an assumption — a missing file, a slow network, a bad value. **Why errors exist instead of just returning a special value**: an error can't be silently ignored the way a return value can. If you don't handle it, it travels up and crashes loudly — which is actually a good thing, because a loud crash is easier to notice and fix than a silent wrong answer. **When to catch an error**: only when you

can actually do something useful about it — retry, fall back to something else, or give the caller a clearer message. **When not to:** don't catch an error just to make it disappear. `except: pass` is exactly how real bugs stay invisible until a user reports something with no logs to explain it. **How it works:** Python's errors form a family tree (`Exception` is the base most custom errors come from). Catching the broad `Exception` is usually still too broad — catch the *specific* error you know how to handle. A custom error, like `MissingConfigError(Exception)`, lets other code catch exactly that one problem without accidentally hiding something unrelated.

Virtual environments: keeping projects separate

A virtual environment ("venv") is a separate, isolated copy of Python just for one project — its own folder of installed packages, kept apart from your system Python and every other project's venv. **Why it exists:** sooner or later, two projects on the same computer will need different, incompatible versions of the same package. Without separation, installing one project's packages can quietly break another project. **When to make one:** at the very start of every project, before installing anything. **How it works:** `python -m venv .venv` creates the isolated folder. Running `source .venv/bin/activate` tells your terminal to use that folder's Python and pip instead of the system ones. `pip freeze > requirements.txt` writes down exactly what's installed, so anyone (including future you) can rebuild the same setup with `pip install -r requirements.txt`. The `.venv` folder itself never gets added to git — only the requirements file does, because the venv can be rebuilt any time, and its contents can be large.

`.env` files: keeping secrets out of your code

An environment variable is a piece of information (like a password or API key) that lives outside your code, in the running program's environment, and is read with `os.getenv()`. **Why this matters:** secrets like API keys must never be typed directly into your code and committed to git. Git history basically lasts forever, and a private repo today might get shared or leaked tomorrow — with the secret still sitting in an old commit. A `.env` file holds these secret values on your own computer. It gets loaded at startup (usually with `python-dotenv`'s `load_dotenv()`), and — this part matters — it's listed in `.gitignore` so it never gets committed. **When to use this:** any value that's different on your computer vs. a teammate's, or that would cause harm if it leaked, belongs in an environment variable, not in code. **How a good config loader should work:** check that every required value is present the moment your program starts, and fail immediately with a clear message naming exactly what's missing. Don't let a missing secret show up later as some confusing error three functions deep — like a strange "401 Unauthorized" from OpenAI, when the real problem was a typo in your `.env` file.

Logging: better than `print()`

`print()` just writes text to the screen — no severity level, no timestamp, and no way to turn it off later without editing your code. A **logger** gives you levels (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`), so you can leave detailed messages in your code forever, and just control — through

settings, not code changes — how much of it you actually see at any given time. **Why this matters for anything that runs on its own** (a server, a scheduled job, an agent): when it fails at 3 AM, `print()` output that scrolled off a terminal that no longer exists is just gone. Log lines written to a file (or sent to a logging service) are how you piece together what actually happened. **When to use logging:** from the very first line of any code that runs outside your own terminal session. **How it works:** Python's built-in `logging` module gives every file its own named logger (`logging.getLogger(__name__)`), so every log line shows exactly where it came from. "Handlers" decide *where* logs go (screen, file, remote service) separately from *what* gets logged — which is why the same code can stay quiet in production and get noisy while you're debugging, just by changing settings, not code.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [Python official tutorial](#) — functions, modules, errors, classes. Skim what you know, read closely what you don't.
- [typing module docs](#) — type hints, and why they're more than "nice to have."
- [Errors tutorial](#) — the official docs on try/except/raise.
- [Real Python — Python Logging](#) — proper logging vs. `print()`.
- [Real Python — Virtual Environments Primer](#) — why venvs exist, how to use one.
- [The Twelve-Factor App — Config](#) — why secrets and settings never live in code. Short read — worth re-reading before Doc12 too.

Practice Exercises (before the build task)

Setup for this document's practice code: work inside `01_python_foundations/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install python-dotenv pydantic`.

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py`, with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python env_config_practice.py`.

For this document, save your practice code as: - **Basic** (typed function + custom error) and **Failure handling** (three different errors, three reactions) are both about writing and handling your own custom errors — save them together as `custom_errors_practice.py`, one section per level. - **Basic (part 2)** (a real venv, start to finish) is its own topic — save it as `venv_setup_practice.py`. - **Intermediate** (read `.env` by hand) and **Edge cases** (empty `.env`

values) are both about `.env` config — save them together as `env_config_practice.py`, one section per level. - **Intermediate (part 2)** (a logger with two output levels) is its own topic — save it as `logging_practice.py`. - **Real-world** (wire config + logging together) is its own topic — save it as `config_logging_wiring_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Basic part 2](#) · [Intermediate](#) · [Intermediate part 2](#) · [Real-world](#) · [Edge cases](#) · [Failure handling](#) · [Build Task](#)

Basic — typed function + custom error

- **What:** one function with full type hints, and one custom error class it raises on bad input.
- **Why:** this is the single habit most beginner code is missing — a function whose signature already tells the reader what can go wrong, before they read the body.
- **When you'll hit this for real:** every time you write a function another file will import. In this document's own Build Task below, `load_config()` does exactly this — it refuses to hand back a broken config instead of failing later, confusingly, three functions away.
- **How to code it:** write `class InvalidAgeError(Exception): pass`. Write `def set_age(age: int) -> None:` that raises `InvalidAgeError` if `age < 0`. Call it once with a valid age, once with a negative one inside `try/except InvalidAgeError`, and print what happened each time.
- **Save as:** `custom_errors_practice.py`, under a `# Basic` section (this file also holds the Failure handling exercise below, in its own `# Failure handling` section).
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Basic (part 2) — a real venv, start to finish

- **What:** create a virtual environment, install one real package, freeze it.
- **Why:** this is the very first thing you do on every single project from here on — get it into muscle memory now, not while also trying to debug something else later.
- **When you'll hit this for real:** literally the "Setup" step at the top of every project in this curriculum.
- **How to code it:** `python -m venv .venv` → `source .venv/bin/activate` → `pip install requests` → `pip freeze > requirements.txt` → open `requirements.txt` and confirm `requests` is listed with a version number.
- **Save as:** `venv_setup_practice.py`.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — read `.env` by hand, then explain why not to

- **What:** parse a `.env` file's `KEY=value` lines yourself, without `python-dotenv`.
- **Why:** doing it by hand once shows you exactly what the library saves you from (quoting, blank lines, comments, missing files) — a shortcut you should still understand under the hood.
- **When you'll hit this for real:** any time you're debugging *why* a `.env` value isn't loading — knowing the manual version means you can check each step yourself instead of treating the library as a black box.
- **How to code it:** open the file, read it line by line, split on the first `=`, and build a dict. Then rewrite the same thing in 2 lines using `python-dotenv`'s `load_dotenv()` + `os.getenv()`. Write one sentence on which version you'd actually ship.
- **Save as:** `env_config_practice.py`, under an `# Intermediate` section (this file also holds the Edge cases exercise below, in its own `# Edge cases` section).
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate (part 2) — a logger with two output levels

- **What:** a logger that prints `INFO` and above to the screen, but writes `DEBUG` and above to a file.
- **Why:** this is the exact setup you want in production — quiet in the terminal, detailed in the file you check after something breaks.
- **When you'll hit this for real:** this document's Build Task `get_logger()` function needs exactly this, and every later project reuses it.
- **How to code it:** create a logger with `logging.getLogger(__name__)`, add a `StreamHandler` set to `INFO`, add a `FileHandler` set to `DEBUG`, and log one message at each level (`.debug()`, `.info()`, `.warning()`) to see the difference.
- **Save as:** `logging_practice.py`.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — wire config + logging together

- **What:** a 3-file mini project (`config.py`, `logging_setup.py`, `main.py`) where `main.py` imports both and prints one log line using a value loaded from config.
- **Why:** config and logging are almost always used *together* — this is a small rehearsal of the real Build Task below, so the real one has no surprises.
- **When you'll hit this for real:** this exact pattern (import config, get a logger, use both together) is the first three lines of every script you'll write for the rest of this curriculum.
- **How to code it:** `main.py` calls `load_config()`, then `get_logger(__name__)`, then logs `f"Loaded config for {config.some_field}"` at `INFO` level. Run it and confirm the line appears on screen.
- **Save as:** `config_logging_wiring_practice.py`.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Show me the solution](#)

Edge cases — is an empty value "missing" or "valid"?

- **What:** two tricky `.env` states: a file that exists but is completely empty, and a key present with an empty string as its value.
- **Why:** this is a real judgment call every config loader has to make, and getting it wrong silently is how a genuinely missing secret sails through as if it were fine.
- **When you'll hit this for real:** someone on a team copies `.env.example` to `.env`, forgets to fill in a value, and your app has to decide right then whether that's a hard failure or not.
- **How to code it:** test your Doc01 config loader against both cases directly. For each, write down — in a comment or a one-line note — whether your loader currently treats it as "missing" (raises `MissingConfigError`) or "valid" (returns an empty string), and whether that's actually the behavior you want.
- **Save as:** `env_config_practice.py`, under an `# Edge cases` section (this file also holds the Intermediate exercise above, in its own `# Intermediate` section).
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure handling — three different errors, three different reactions

- **What:** one function that raises 3 different kinds of errors depending on the input, and a caller that handles each one differently — not one big catch-all `except`.
- **Why:** a single blanket `except Exception` treats a typo'd input the same as a real system failure — this exercise builds the habit of catching each error type separately, so your response actually fits the problem.
- **When you'll hit this for real:** any function with more than one way to fail differently — like Doc04's chat client, which needs to react differently to a bad key vs. a rate limit vs. a network timeout.
- **How to code it:** define 3 small custom error classes, write one function that raises a different one depending on its input, then write 3 separate `except` blocks (not one shared one) that each print something different.
- **Save as:** `custom_errors_practice.py`, under a `# Failure handling` section (this file also holds the Basic exercise above, in its own `# Basic` section).
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Config & Logging Foundation

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: a reusable `config` and `logger` setup that every later project will import.

Requirements:

- Loads required settings (at least `OPENAI_API_KEY`) from a `.env` file.

- Fails fast with a clear, custom error if something required is missing — never quietly uses a fake default for a secret.
- Gives you a typed config object (not a plain dictionary), so your editor can help with autocomplete and catch typos.
- Has a `get_logger(name)` function that returns a ready-to-use logger (at least a console handler; level set by an environment variable).

Inputs: a `.env` file (you create it), and a `.env.example` with no real values.

Outputs: a config object and a logger that any script in this project can import.

Constraints:

- No secrets typed directly in code — not even in `.env.example`.
- No bare `except: .`
- Every function has full type hints.

Suggested files:

```
01_python_foundations/
├─ config.py
├─ logging_setup.py
├─ exceptions.py
├─ .env.example
└─ test_config.py
```

Functions/Components to build (you write the code):

- `exceptions.py` → `MissingConfigError(Exception)`
- `config.py` → `load_config() -> Config` (a dataclass or similar)
- `logging_setup.py` → `get_logger(name: str) -> logging.Logger`

Expected Behavior

- Missing `.env`, or a missing key inside it → `MissingConfigError`, with a message naming exactly what's missing — not a generic `KeyError`.
- Valid `.env` → `load_config()` returns a working object. Accessing a field that doesn't exist should be caught by your editor/type-checker, not surprise you while running.
- Calling `get_logger("my_module")` from two different files should produce log lines you can tell apart (the module name should be visible).

Test Cases

Scenario	Expected
<code>.env</code> missing entirely	<code>MissingConfigError</code> , message names the missing file
<code>.env</code> exists, but <code>OPENAI_API_KEY</code> is missing	<code>MissingConfigError</code> , message names the missing key
<code>.env</code> exists and is valid	<code>Config</code> object returned, <code>config.openai_api_key</code> works
<code>get_logger("x")</code> called twice with the same name	Works both times, no duplicate log lines per message

Break-It / Debug Preview

- An empty `.env` file (exists, but has nothing in it).
- `.env` has the key, but its value is an empty string — is that "missing" or "valid"? Decide, and be ready to defend your answer.
- Full guided debugging happens in [14_debugging_lab](#).

Interview Topics Preview

- `venv` vs. installing globally · mutable default arguments · the error family tree · why logging beats `print()` in production · what `__init__.py` does.

Move On When

You can build a small, multi-file Python project — with logging and config — from scratch, without being told the file layout. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** (a concept) up through **Hint 4** (a small code snippet). Ask for the full solution only if you say "**Show me the solution.**"